SEN4100 – Software Project Management

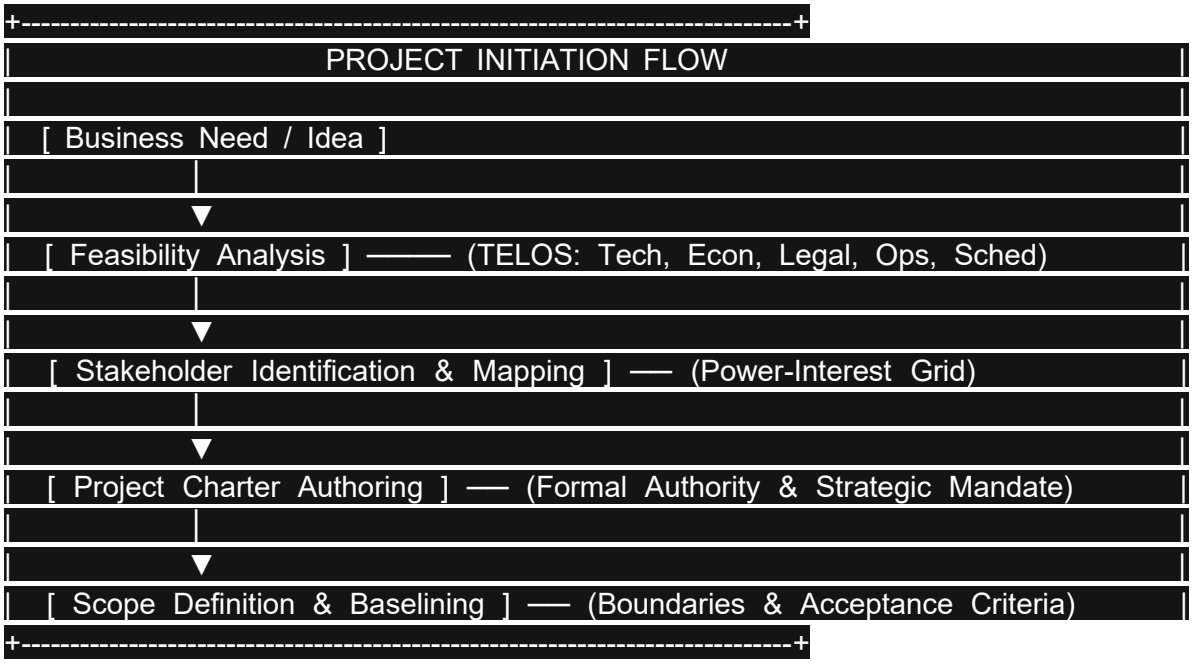
Week 3 Lecture Notes: Project Scope, Requirements & Feasibility

Course Information & Academic Framework

- **Course Instructor:** Asst. Prof. M. Fatih Mintaş
- **Course Learning Outcomes (CLOs) Addressed:**
  - Understand the fundamentals of managing a software project from initiation to closure.
  - Prepare a comprehensive project plan addressing scope, schedule, cost, and quality baselines.
  - Evaluate project feasibility and apply systematic change management to prevent scope degradation.
- **Primary Textbooks:** Pressman & Maxim (*Software Engineering: A Practitioner's Approach*), Hughes & Cotterell (*Software Project Management*), Ian Sommerville (*Software Engineering*).

1. Foundations of Project Scope & The Initiation Phase

Project Initiation is the formal gateway wherein an abstract business need, organizational problem, or market opportunity is evaluated and converted into an authorized engineering project. Within software engineering, failure to establish unambiguous boundaries during initiation is the leading contributor to project abandonment.



1.1 Defining Scope: Product Scope vs. Project Scope

A software project manager must maintain a clear distinction between the deliverable itself, and the operational effort required to construct it:

- **Product Scope:** The full set of features, capabilities, operational functions, data models, and non-functional characteristics that characterize the software deliverable itself.
  - *Verification Metric:* Measured against the Software Requirements Specification (SRS), product backlog acceptance criteria, and system verification test suites.
  - *Example:* "The microservice must authenticate users via OAuth 2.0 / OpenID Connect and process 1,500 transactions per second at sub-50 ms latency."
- **Project Scope:** The entirety of work, engineering activities, governance milestones, tool chain configurations, documentation, and coordination required to deliver the specified product scope.
  - *Verification Metric:* Measured against the Project Management Plan, Work Breakdown Structure (WBS), and schedule baseline.
  - *Example:* "Provisioning Kubernetes staging clusters, establishing automated CI/CD security scanning, authoring user manuals, and executing load testing."

2. Feasibility Analysis & Economic Evaluation Models

Before capital expenditure is authorized, an organization must conduct a structured feasibility study to ensure the software endeavor is viable across multiple dimensions.

2.1 The TELOS Framework

The **TELOS** framework establishes five structural evaluation vectors:

Dimension	Core Analytical Question	Key Risk Factors
T – Technical	Do we possess the technical capabilities, architecture, and tools to build this?	Unproven frameworks, bleeding-edge AI models, concurrency bottlenecks, lack of internal expertise.
E – Economic	Does the anticipated financial return or cost saving justify development costs?	Capital outlay, cloud egress costs, third-party API licensing, underestimating engineering hours.
L – Legal & Regulatory	Does the software comply with national statutes, data protection, and IP laws?	GDPR / KVKK compliance, medical device certification (FDA/IEC 62304), open-source licensing (GPL vs. MIT).

O – Operational	Will the end-users and organizational culture adopt the system post-deployment?	User resistance, workflow disruption, training deficits, alignment with organizational structure (Conway's Law).
S – Schedule	Can the software realistically be engineered and delivered within market deadlines?	Imposed commercial release dates, hardware delivery dependencies, critical path fragility.

2.2 Algorithmic Financial Feasibility Metrics

Economic feasibility requires objective mathematical modeling of capital expenditure against projected operational cash flows.

1. Payback Period (PBP)

The time required for cumulative net cash inflows to equal the initial capital investment $\boldsymbol{I_0}$:

$$\boldsymbol{Cumulative\ Cash\ Flow\ (T) = I_0}$$

- *Limitation:* Ignores the time value of money and cash flows generated after the payback point.

2. Return on Investment (ROI)

A standardized percentage reflecting the overall profitability of software expenditure:

$$ROI = \frac{\sum Total\ Benefits - \sum Total\ Costs}{\sum Total\ Costs} * 100\%$$

3. Net Present Value (NPV)

The definitive financial metric for capital budgeting, evaluating cash inflows and outflows by discounting them back to their present value using a specified discount rate (r):

$$NPV = \sum_{t=0}^{n} \frac{CF_t}{1 + r^t}$$

Where:

- CF_t = Net cash flow generated at time t (inflows minus outflows).
- r = Required rate of return, cost of capital, or hurdle rate.
- t = Time period (usually measured in years).
- n = Total operational lifecycle horizon of the software system.

Decision Rule:

- If $NPV > 0$: The project yields return greater than the cost of capital; **Accept the project**.
- If $NPV < 0$: The project destroys capital value; **Reject the project**.
- When evaluating competing projects: Select the initiative with the highest positive NPV.

Worked Calculation Example:

An engineering firm evaluates an internal automated test-infrastructure project requiring an upfront capital outlay of **$100,000** at year 0. The automation delivers operational wage savings of **$40,000** annually across Years 1, 2, and 3. The organization's cost of capital r is **8% (0.08)**:

$$NPV = -100{,}000 + \frac{40{,}000}{(1+00.8)^1} + \frac{40{,}000}{(1+00.8)^2} + \frac{40{,}000}{(1+00.8)^3}$$

$$NPV = -100{,}000 + \frac{40{,}000}{1.08} + \frac{40{,}000}{1.1664} + \frac{40{,}000}{1.2597}$$

$$NPV = -100{,}000 + 37{,}037.04 + 34{,}293.55 + 31{,}753.29$$

$$NPV = -100{,}000 + 103{,}083.88 = +\$3{,}083.88$$

Verdict: Since $NPV = +\$3{,}083.88 > 0$, the project is economically viable.

3. Stakeholder Identification, Analysis & Engagement

A **stakeholder** is any individual, group, or organization that can affect, be affected by, or perceive itself to be affected by a decision, activity, or outcome of the software project.

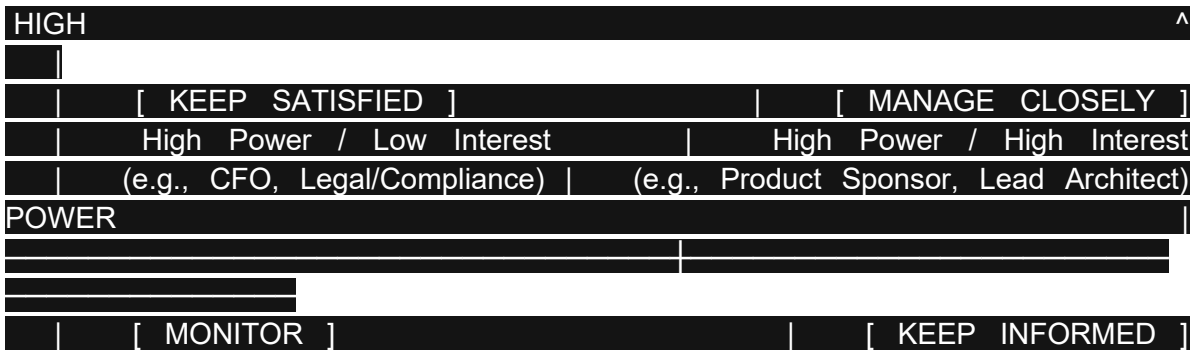
3.1 The Stakeholder Register

The Stakeholder Register is created during initiation to systematically catalog all project participants and impacted parties:

- **Identification Data:** Name, department, organizational role, project role (e.g., Executive Sponsor, Product Owner, Lead Architect, Compliance Auditor).
- **Assessment Data:** Major expectations, operational pain points, influence level, critical dependencies.
- **Stakeholder Classification:** Internal vs. External; Supporter, Neutral, or Resistor.

3.2 The Power-Interest Matrix (Mendelow's Grid)

Stakeholder management strategies are governed by mapping stakeholders across two axes:





- **High Power / High Interest (Manage Closely):** Primary partners. Must be actively engaged in architectural decisions, sprint reviews, and steering committees.
- **High Power / Low Interest (Keep Satisfied):** Institutional authority figures. Keep them content with high-level summaries and governance compliance; avoid overwhelming them with granular engineering details but never let them be surprised by budget or legal shifts.
- **Low Power / High Interest (Keep Informed):** End-users and operational staff. Highly invested in user experience and functionality; consult them during requirements elicitation to ensure operational adoption.
- **Low Power / Low Interest (Monitor):** Minimum effort required; track via periodic automated reports or changelogs.

4. The Project Charter: Authorization & Governance

The **Project Charter** is the foundational governance document that formally authorizes the existence of a project and vests the Software Project Manager with organizational authority to assign enterprise resources to engineering activities.

4.1 Structural Components of a Project Charter

A comprehensive Project Charter must contain:

1. **Project Title & Administrative Metadata:** Formal code, version, approval dates, project sponsor, assigned SPM.
2. **Business Case & Strategic Justification:** The organizational problem being addressed, strategic market opportunity, and financial ROI/NPV projections.
3. **Measurable Project Objectives & Success Criteria:**
   - Specific, Measurable, Achievable, Relevant, Time-bound (SMART) criteria.
   - Key Performance Indicators (KPIs) (e.g., "Reduce average customer check-out processing time from 42 seconds to under 8 seconds within 6 months of rollout").
4. **High-Level Requirements & Boundaries:** Primary functional capabilities and explicit system boundaries.
5. **Assigned Project Manager & Authority Limits:** Statement delineating SPM's authority (e.g., hiring contractors, signing off on budget variances up to $15,000, selecting technical frameworks).
6. **Key Stakeholders & Governance Committee:** Sponsor signatures, Product Owner assignment, core steering group.
7. **Summary Milestone Schedule:** Initial delivery gates (e.g., Architecture Freeze, Prototype Complete, Beta Deployment, Final UAT).
8. **Estimated Budget & Financial Baseline:** Top-down capital authorization limits.
9. **Known High-Level Risks:** Macro technical and business threats identified during preliminary feasibility.

10. **Assumptions and Constraints:**
    - *Assumptions:* Factors assumed to be true without absolute proof (e.g., "Third-party payment gateway APIs will maintain 99.95% uptime").
    - *Constraints:* Rigid limitations imposed by external boundaries (e.g., "The production deployment must complete before statutory compliance deadlines on November 1").

Critical Rule for SPMs: Never commence technical architecture or sprint execution without an officially signed Project Charter. Developing software without a signed charter leaves the engineering team vulnerable to unbudgeted cancellation and unauthorized scope claims.

5. Scope Definition, Baselining & Requirements Engineering

Once the charter is signed, high-level business goals must be refined into an actionable **Scope Baseline**.

5.1 The Scope Statement

The **Project Scope Statement** documents the entire scope, including project deliverables, major assumptions, and operational constraints. It serves as the single source of truth between clients and engineering squads:

- **Product Scope Description:** Granular elaboration of system capabilities.
- **Deliverables:** Tangible work products (e.g., deployable Docker containers, REST API schemas, database migration scripts, test reports, architecture documentation).
- **User Acceptance Criteria:** Explicit qualitative and quantitative conditions that must be fulfilled before the client legally signs off on deliverables.
- **Explicit Scope Exclusions ("Out of Scope"):** Equally critical as inclusions. Stating what the project will **not** build (e.g., "The MVP will not support multi-currency processing or offline desktop sync") prevents immediate stakeholder misalignment.

5.2 The Triple Components of the Scope Baseline

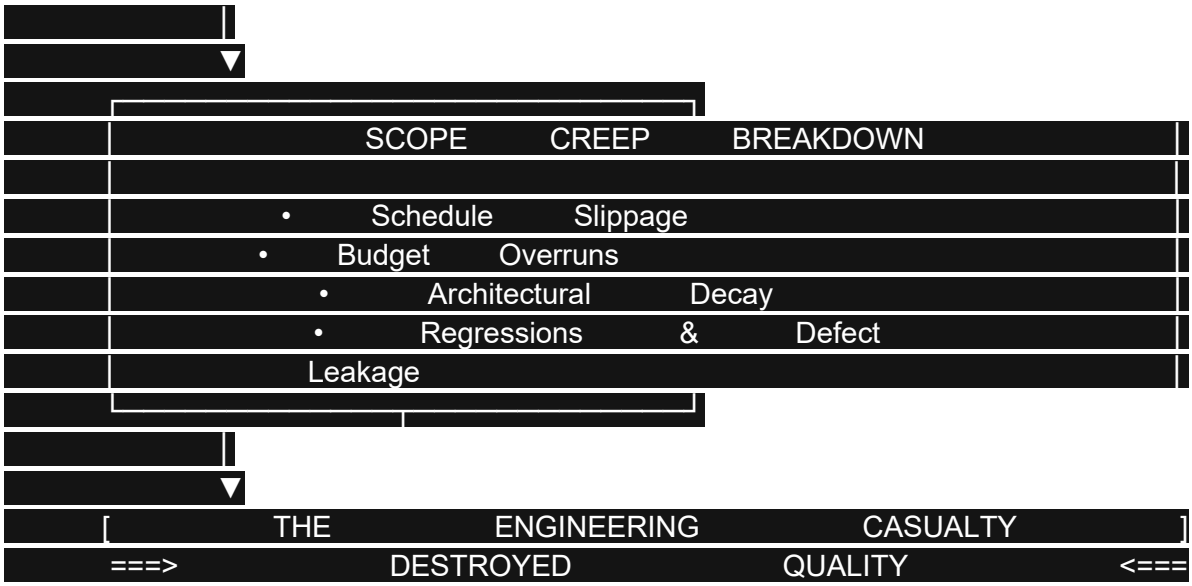
In formalized project management (PMBOK standard), the Scope Baseline consists of:

1. **The Approved Project Scope Statement.**
2. **The Work Breakdown Structure (WBS)** (Decomposing deliverables into discrete work packages).
3. **The WBS Dictionary** (Detailed descriptions of each work package, milestones, and acceptance attributes).

6. The Pathology of Scope Creep & Change Control Governance

Scope Creep refers to the uncontrolled, incremental expansion of project or product scope without corresponding adjustments to time, cost, and human resources.

UNCONTROLLED	CHANGE	INPUTS
(Informal	Client	Requests,
"Quick"	Feature	Tweaks,
Developer		Perfectionism)

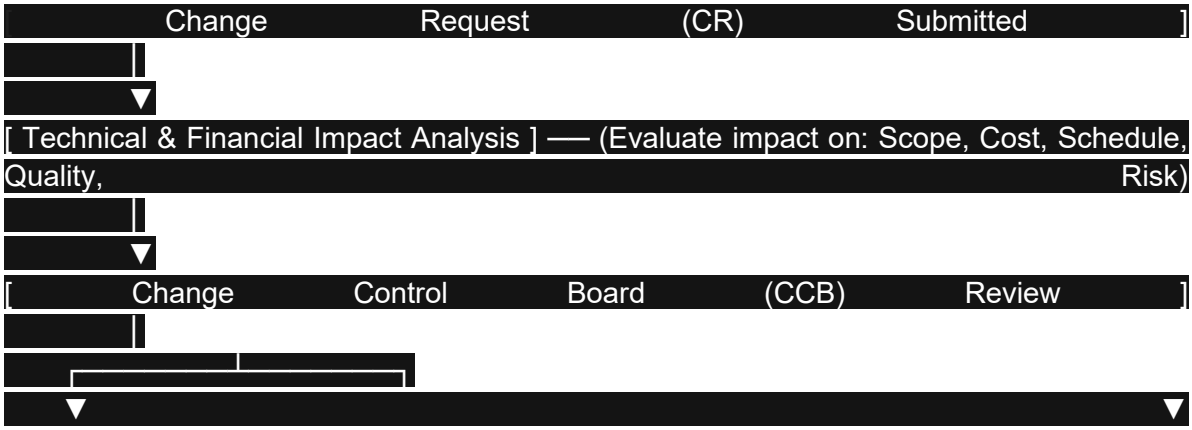


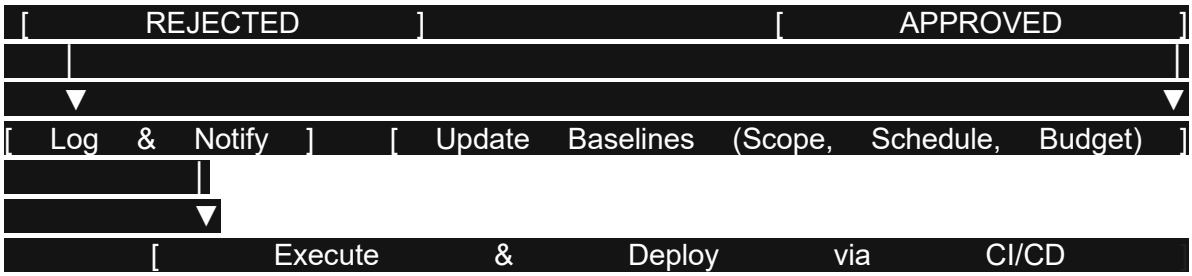
6.1 Root Causes of Scope Creep in Software Engineering

1. **Ambiguous Requirements:** Vague statements such as "the system must be user-friendly and fast" invite varying interpretations.
2. **Direct Informal Communications:** Developers agree to "minor adjustments" requested directly by clients over informal channels (Slack, email) without logging into them.
3. **Gold Plating:** Developers independently adding unrequested features, complex abstractions, or unneeded optimizations because they find them technically interesting.
4. **Lack of a Formal Baseline:** If an initial baseline was never frozen, scope cannot be tracked or controlled.

6.2 The Formal Change Control Workflow

To manage changes without stifling legitimate business adaptation, a disciplined **Change Control System** must be maintained:





- **The Change Control Board (CCB):** A formal committee consisting of the Project Sponsor, Project Manager, Lead Architect, and Customer Representative.
- **Impact Analysis:** Before the CCB convenes, the SPM and technical leads document the exact impact of the requested change:
  - How many additional story points / engineer-hours are required?
  - What is the critical path impact on the milestone schedule?
  - Does the change destabilize existing architectural modules or introduce new security vectors?
  - What trade-off must occur (e.g., if we accept Feature X, what lower-priority feature drops out of the sprint backlog)?

7. Real-World Case Studies

Case 1: Denver International Airport (DIA) Automated Baggage System

- **The Incident:** An ambitious, highly automated computerized baggage-handling system involving hundreds of autonomous carts, barcode scanners, and distributed tracking networks was scheduled to open in 1993. The project was delayed by 16 months, costing the city roughly $560 million in bond interest and operational losses before being scaled back and ultimately abandoned.
- **Scope & Initiation Pathology:**
  - The city attempted to impose a single, massive automated system across all airlines without accounting for fundamentally different airline operational requirements (e.g., United Airlines' hub operations vs. point-to-point carriers).
  - System scope was expanded mid-flight to include complex routing modifications without recalculating physical and architectural throughput limits.
  - Time and budget constraints were fixed by political municipal deadlines while scope escalated continuously, causing severe electromechanical and software interface failures during integration testing.
- **SPM Lesson:** A project manager must defend the Iron Triangle. Expanding scope without extending schedule or increasing system capacity leads to systemic project failure.

Case 2: The FBI Virtual Case File (VCF) System

- **The Incident:** Initiated post-9/11 to replace outdated legacy IT systems with modern criminal case tracking. After spending $170 million between 2001 and 2005, the entire project was cancelled with zero functional code deployed.
- **Scope & Governance Pathology:**
  - Over 800-page specification binders were authored, but requirements were modified

repeatedly without centralized configuration or change management.
    - Between 2001 and 2004, the FBI changed project directors five times, resetting project priorities and introducing massive scope creep.
    - Contractors wrote hundreds of thousands of lines of fragile procedural code before realizing the architecture could not support transactional security and distributed access constraints.
- **SPM Lesson:** Upfront requirements must be translated into controlled, verified baselines. Without a stable Project Charter, clear authority limits, and a disciplined Change Control Board, multi-million-dollar software systems collapse under bureaucratic scope drift.

8. Applied Alignment with Course Term Project

The topics covered this week form the structural core of your **Milestone 1 Deliverable** (due in Week 4, accounting for **5%** of your overall course grade):

- **Mandatory Deliverable Components:**

1. **Executive Project Charter:** Authored according to the professional template presented today (including business case, measurable KPIs, assigned squad roles, assumptions, and constraints).
2. **Stakeholder Register & Power-Interest Grid:** Identification of at least 6 critical project stakeholders mapped across Mendelow's matrix.
3. **Formal Scope Statement:** Explicit itemization of Product Scope, Project Scope, Acceptance Criteria, and specific Out-of-Scope exclusions.
4. **Preliminary Feasibility Review:** A documented TELOS analysis justifying technical viability and economic logic.

- **Jira / GitHub Projects Setup:** Every squad must establish a centralized backlog repository reflecting the scope baseline agreed upon in the Project Charter.

9. Classroom Discussion Scenarios & Review Questions

1. **The "Small Favor" Trap:**
    - A prominent corporate client calls your junior backend developer directly and requests: *"Could you quickly add a PDF export button to the reporting view? It's just a simple 2-hour task."* The developer obliges without informing you.
    - *Question:* How does this action violate project governance? What systemic risks does this create for testing, documentation, and the baseline? How should the developer have responded?
2. **Managing Negative NPV Strategic Projects:**
    - Under what organizational circumstances would an enterprise executive committee intentionally approve a software project with a calculated negative Net Present Value ($\boldsymbol{NPV < 0}$)?
    - *Consideration:* Regulatory mandates (GDPR fines), strategic platform investments, life-safety requirements, or creating architectural prerequisites for future high-value applications.
3. **Agile vs. Predictive Scope Baselining:**
    - How does the concept of a "Scope Baseline" differ between a plan-driven project governed by an SRS and an adaptive Scrum team governed by an evolving Product Backlog?

10. Next Week Preview & Assigned Reading

- **Week 4 Topic:** Project Planning & Work Breakdown Structure (WBS).
- **Core Agenda:** Decomposition principles, the 100% Rule, Work Packages, Deliverables, Milestones, and authoring the WBS Dictionary.
- **Assigned Reading:**
  - Pressman & Maxim: Chapter 5 (*Understanding Requirements*) and Chapter 6 (*Requirements Modeling*).
  - Hughes & Cotterell: Chapter 3 (*Programme Management and Project Evaluation*).
- **Reminders:** Squad registrations must be completed; Milestone 1 draft consultations will take place during scheduled office hours.